

Lab: Composable Functions, Recomposition, and LazyList in Jetpack Compose

Abbreviations used in this document

Abbreviation	Full term
UI	User Interface
API	Application Programming Interface
SDK	Software Development Kit
DSL	Domain-Specific Language
XML	Extensible Markup Language
JSON	JavaScript Object Notation

Based on: <https://developer.android.com/develop/ui/compose/mental-model> **You will need:** Android Studio with an Android SDK installed, and a running emulator or physical device — no other setup. **Projects used in this lab:** `Display0`, `IncreasingCounter`, `ChangeName` (imperative/Views) and `JetPackDisplay0`, `JetPackIncreasingCounter`, `JetPackChangeName`, `JetPackListPeople` (declarative/Compose) — all under `MobileProgramming26/labs/`. Each project folder has its own README with build/run instructions.

By the end of this lab you will have compared the imperative (Views) and declarative (Compose) way of building the same three simple apps, traced exactly how and when Compose recomposes, defined a Kotlin `data class` and converted a list of them to/from JSON, and used `LazyColumn` to display a scrollable list.

Table of contents

- [Learning objectives](#)
- [1. From imperative to declarative](#)
 - [1.1 The problem with the imperative model](#)
 - [1.2 The declarative model](#)
 - [1.3 Recap](#)
- [2. Composable functions: anatomy and rules](#)
 - [2.1 Anatomy of a composable function](#)
 - [2.2 The three golden rules](#)
 - [2.3 A snippet to think about](#)
- [3. The simplest composable: `JetPackDisplay0`](#)
- [4. State and recomposition: the counter](#)
 - [4.1 The imperative way: `IncreasingCounter`](#)
 - [4.2 The declarative way: `JetPackIncreasingCounter`](#)
 - [4.3 `remember` and `mutableIntStateOf`](#)
 - [4.4 Tracing a recomposition](#)
 - [4.5 Why doesn't the whole function "reset"?](#)

- 5. Two independent pieces of state: changing a name
 - 5.1 The imperative way: `ChangeName`
 - 5.2 The declarative way: `JetPackChangeName`
 - 5.3 Two independent state variables
 - 5.4 The central idea
 - 6. Golden rules of recomposition
 - 6.1 Never rely on side effects
 - 6.2 Recomposition is optimistic and can be canceled
 - 6.3 Composable functions run often — avoid expensive work
 - 6.4 Never write to local variables during composition
 - 6.5 Execution order isn't guaranteed
 - 7. Summary
 - 8. Kotlin data types, and building a `JSONArray` of people
 - 8.1 Basic Kotlin data types
 - 8.2 `data class`: modeling one record
 - 8.3 From a `List<Person>` to a `JSONArray`
 - 8.4 Going back: `JSONArray` to `List<Person>`
 - 9. Introducing `Lazy` / `LazyColumn`
 - Additional material
 - A1. The Observer pattern
 - A2. The Delegation pattern and Kotlin's `by`
 - A3. The same three apps in vanilla JavaScript
 - A4. The same three apps in React
 - A5. The same three apps in SwiftUI
 - A6. The same three apps in Flutter
 - A7. `LazyColumn`/`LazyRow` and the `key` parameter
-

Learning objectives

By the end of this material, you should be able to:

1. Explain the difference between the imperative model (traditional Views) and the declarative model (Compose).
 2. Write a simple `@Composable` function and explain its rules (fast, idempotent, side-effect free).
 3. Explain what triggers a recomposition and why Compose can "skip" parts of the UI tree.
 4. Identify incorrect code (with side effects) inside a composable and fix it.
 5. Define a `data class` for a simple record and convert a `List` of them to/from a `JSONArray/JSONObject`.
 6. Use `LazyColumn` to display a list, and explain why (and how) its recomposition is scoped per-row rather than to the whole list.
-

1. From imperative to declarative

1.1 The problem with the imperative model

In the classic Android model (Views/XML), the UI is a tree of stateful widgets. Updating it requires:

- Finding the widget with `findViewById()`.
- Calling mutator methods: `button.setText(...)`, `img.setImageBitmap(...)`, `container.addChild(view)`.

Live example: **Display0** (the imperative-Views version of **JetPackDisplay0**)

File: `Display0/app/src/main/res/layout/activity_main.xml`

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/numberText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="0"
        android:textSize="72sp" />

</FrameLayout>
```

File: `Display0/app/src/main/java/com/example/display0/MainActivity.kt`

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Display0 and **JetPackDisplay0** produce the **exact same result**: a "0" centered on screen at 72sp. But the way they get there is fundamentally different:

- Here, the UI is defined in a separate XML file (`activity_main.xml`) as a **tree of widget objects** (`FrameLayout`, `TextView`) that get instantiated and kept alive in memory.
- `setContentView(R.layout.activity_main)` inflates that XML into real **View** objects once, at startup.
- If we wanted this "0" to change later, we would need to grab a reference to the widget — `findViewById<TextView>(R.id.numberText)` — and imperatively call `numberText.text = newValue`. The layout file itself never changes; we'd be mutating the live widget object.

Problems with this approach:

- It's easy to forget to update a view if the same data is shown in multiple places (e.g. two different `TextViews` both showing "0").
- The order of updates can cause illegal or inconsistent states.
- Complexity grows fast with the number of views — manually keeping "state → UI" in sync by hand doesn't scale.

Discussion question: Have you ever had a bug where part of the screen didn't update because you forgot to call a `setText`? Or where two views showed contradictory data?

1.2 The declarative model

The industry (React, Flutter, SwiftUI, Compose) has shifted to a declarative model:

- Instead of mutating the UI step by step, **you describe how the UI should look for a given state**.
- Conceptually, Compose "regenerates the entire screen from scratch" every time state changes.
- Since regenerating everything would be computationally expensive, Compose applies **intelligent recomposition**: it only re-executes the parts that actually depend on the data that changed.

Data flow diagram:

```

App logic (ViewModel / state)
    ↓
Top-level composable function
    ↓
Child composable functions
    ↓
UI elements on screen
  
```

Flow when the user interacts:

```

User interaction (tap, text input, etc.)
  → UI event (onClick, onValueChange...)
  → App logic updates the state
  → Compose calls the affected composable functions again
  → The UI is "redrawn" (recomposition)
  
```

1.3 Recap

Comparison of `Display0` vs. `JetPackDisplay0`:

	Imperative (<code>Display0</code>)	Declarative (<code>JetPackDisplay0</code>)
Where the UI is defined	<code>activity_main.xml</code> (separate file)	<code>DisplayZero()</code> function (Kotlin code)
Widgets	Stateful <code>View</code> objects, expose getters/setters (<code>numberText.text = ...</code>)	Relatively stateless — no getter/setter functions
Updates	You'd mutate the widget by hand via <code>findViewById</code>	You describe the UI based on current state; Compose redraws it
Architecture	Direct widget manipulation	State flows down, events flow up

2. Composable functions: anatomy and rules

2.1 Anatomy of a composable function

```
@Composable
fun Greeting(name: String) {
    Text("Hello $name")
}
```

Key points:

1. **The @Composable annotation:** tells the Compose compiler that this function transforms data into UI. It enables the special compiler plugin Compose needs.
2. **It takes data as parameters:** the app logic "passes in" the state it needs to display (`name`).
3. **It calls other composable functions:** `Greeting` composes its UI by calling `Text`. This is how the UI tree is built: composables calling composables.
4. **It doesn't return anything** (implicit `Unit`): a composable doesn't return a widget, it **describes** how part of the UI should look.
5. **It can use Kotlin freely:** `if` statements, `for` loops, helper functions.

```
@Composable
fun Greeting(names: List<String>) {
    for (name in names) {
        Text("Hello $name")
    }
}
```

2.2 The three golden rules

Every composable function should be:

- **Fast:** it may run many times per second (e.g. during an animation), so it must not do expensive work (I/O, network, disk) directly.
- **Idempotent:** calling it multiple times with the same arguments must always produce the same result / the same UI.
- **Side-effect free:** it must not modify global variables, properties of shared objects, `SharedPreferences`, or anything outside its own scope.

Why it matters: if Compose decides to skip or cancel a recomposition (as part of its optimizations), any side effect inside the composable might not run, might run too many times, or might run against an inconsistent state. This is explored further in section 6.

2.3 A snippet to think about

Look at this snippet and ask yourself: does mutating `items` inside this composable violate the rules above? Would you write code like this?

```

@Composable
fun ListWithBug(myList: List<String>) {
    var items = 0
    Column {
        for (item in myList) {
            Text("Item: $item")
            items++ // <-- is mutating a variable here a problem?
        }
    }
    Text("Count: $items")
}

```

Flag the `items++` line as "the kind of thing the side-effect-free rule warns about" and sit with some discomfort about it — section 6.4 revisits this exact snippet and explains what's really going on.

3. The simplest composable: `JetPackDisplay0`

File: `JetPackDisplay0/app/src/main/java/com/example/jetpackdisplay0/MainActivity.kt`

```

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            DisplayZero()
        }
    }
}

@Composable
fun DisplayZero() {
    Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
        Text(text = "0", fontSize = 72.sp)
    }
}

```

Points to notice:

- `setContent { }` is the bridge between the `Activity` world (imperative) and the Compose world (declarative). Everything inside it is a **composable tree**.
- `DisplayZero` is the root composable of the screen: it takes no parameters, has no state, and always renders the same thing. It's the simplest possible example of "describing the UI".
- `Box` and `Text` are composables provided by the library (`androidx.compose.foundation`, `androidx.compose.material3`) — the same pattern you'll use to write your own.
- Since nothing here ever changes, **there is no reason for this screen to ever recompute**: this screen never recomposes after the initial composition, because it doesn't depend on any state.

Run the project and confirm it just displays a fixed "0" centered on the screen.

4. State and recomposition: the counter

4.1 The imperative way: `IncreasingCounter`

File: `IncreasingCounter/app/src/main/java/com/example/increasingcounter/MainActivity.kt`

```
class MainActivity : AppCompatActivity() {

    private var count = 0

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val counterText = findViewById<TextView>(R.id.counterText)
        val addButton = findViewById<Button>(R.id.addButton)

        addButton.setOnClickListener {
            count += 1
            counterText.text = "$count"
        }
    }
}
```

Same app, same behavior as `JetPackIncreasingCounter` — but notice the **mental model** behind this code:

- `count` is a plain field on the `Activity`. Nothing about it is "observed" by anything — it's just a number sitting in memory.
- `counterText` is a **reference to a live widget object**, fetched once via `findViewById`. That object has its own internal state (its current text), independent of `count`.
- The programmer is personally responsible for keeping the two in sync: every time `count` changes, the code must *remember* to also call `counterText.text = "$count"`. If you commented out that line, `count` would silently keep incrementing while the screen kept showing "0" forever.
- There is no "recomposition" concept at all: there's just a click handler that runs exactly the imperative steps it's told to run, in order, on the object graph it was given references to.

Question to consider: what would happen if this screen also had a second `TextView` showing the count in words ("one", "two" ...)? How many places in the code would you need to touch?

4.2 The declarative way: `JetPackIncreasingCounter`

File:

`JetPackIncreasingCounter/app/src/main/java/com/example/jetpackincreasingcounter/MainActivity.kt`

```
@Composable
fun Counter() {
    var count by remember { mutableIntStateOf(0) }
```

```

Column(
    modifier = Modifier.fillMaxSize(),
    verticalArrangement = Arrangement.Center,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text(text = "$count", fontSize = 72.sp)
    Button(onClick = { count += 1 }) {
        Text(text = "Add+1")
    }
}

```

4.3 remember and mutableIntStateOf

- `mutableIntStateOf(0)` creates a state holder that Compose can observe: when its value changes, Compose knows it needs to recompute.
- `remember { ... }` tells Compose: "keep this value across recompositions, don't recreate it every time `Counter()` runs again." Without `remember`, every recomposition would reset `count` back to 0.
- `by` is Kotlin's property delegation: it lets you write `count` and `count = x` directly instead of `count.value`.

This is exactly the cycle described in the article's Recomposition section: *user interaction* → *event* (`onClick`) → *state change* → *Compose calls the composable again* → *the UI redraws*.

Contrast with `IncreasingCounter` (imperative): in the Views version, the programmer manually writes the line that pushes the new value into the widget (`counterText.text = "$count"`). In the Compose version, that line doesn't exist anywhere — `Text(text = "$count", ...)` simply *declares* that this text should always match `count`, and Compose is the one responsible for calling that code again whenever `count` changes. The mental shift is: **stop thinking "when X happens, go update widget Y" and start thinking "this piece of UI is a function of this piece of state — describe that function once."**

4.4 Tracing a recomposition

With each tap on "Add+1", the full cycle is:

```

Tap on the button
→ the lambda onClick = { count += 1 } runs
→ count changes from 0 to 1
→ Compose marks Counter() (and anything reading "count") as "dirty"
→ Compose re-executes the composable Text(text = "$count", ...)
→ the screen shows "1"

```

Important: the `Button` itself **doesn't need to recompute** because it doesn't depend on `count`; only the `Text` that reads `count` recomposes. This is the first concrete evidence of "smart / selective recomposition."

4.5 Why doesn't the whole function "reset"?

If `Counter()` runs again, why doesn't `count` go back to `0`? Think about the role of `remember` here before moving on — then try deleting `remember` in the editor, predict what will happen, and test it.

Hands-on exercise: modify `Counter()` to add a second "Reset" button that sets `count` back to `0`. Before running it, predict which composable recomposes and which doesn't.

5. Two independent pieces of state: changing a name

5.1 The imperative way: `ChangeName`

File: `ChangeName/app/src/main/java/com/example/changename/MainActivity.kt`

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val initialName = "World"

        val greetingText = findViewById<TextView>(R.id.greetingText)
        val nameField = findViewById<EditText>(R.id.nameField)
        val changeButton = findViewById<Button>(R.id.changeButton)

        greetingText.text = "Hello, $initialName"
        nameField.setText(initialName)

        changeButton.setOnClickListener {
            greetingText.text = "Hello, ${nameField.text}"
        }
    }
}
```

Same behavior as `JetPackChangeName`, but look closely at where the "state" actually lives:

- There is no `count`-like variable holding the current name — the **`EditText` widget itself** is the state. Its current text lives inside the live `nameField` object, and the only way to read it is `nameField.text`.
- The greeting is not "kept in sync" with anything: it's set once in `onCreate`, and then only touched again, once, inside the `onClick` listener. If any *other* event could change the name (a second button, a voice-input callback, a saved preference loading late), the programmer would have to remember to duplicate that same `greetingText.text = "Hello, ${nameField.text}"` line inside every one of those places too.
- Notice there's no equivalent of `remember` or `mutableStateOf` at all — because there's no abstraction here that says "this text depends on that value." The dependency between the `EditText`'s content and the greeting only exists in the programmer's head, encoded as an imperative instruction inside one click handler.

Question to consider: if you wanted to add a live character counter below the field (updating on every keystroke, not just on "Change"), what would you need to add here?

5.2 The declarative way: JetPackChangeName

File: JetPackChangeName/app/src/main/java/com/example/jetpackchangenam/MainActivity.kt

```
@Composable
fun ChangeName() {
    val initialName = "World"
    var greetingName by remember { mutableStateOf(initialName) }
    var fieldValue by remember { mutableStateOf(initialName) }

    Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
        Column(
            modifier = Modifier.padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(text = "Hello, $greetingName", fontSize = 24.sp)
            OutlinedTextField(
                value = fieldValue,
                onValueChange = { fieldValue = it },
                modifier = Modifier.padding(top = 16.dp)
            )
            Button(
                onClick = { greetingName = fieldValue },
                modifier = Modifier.padding(top = 16.dp)
            ) {
                Text("Change")
            }
        }
    }
}
```

5.3 Two independent state variables

Notice there are **two** separate `remember { mutableStateOf(...) }` calls:

- **fieldValue**: what the user is currently typing in the `OutlinedTextField` — changes with **every keystroke**.
- **greetingName**: what's shown in the greeting — only changes when the "Change" button is tapped.

This is a perfect demonstration of the article's point: *"recomposition skips as much as possible."*

Live trace, keystroke by keystroke:

```
The user types "J"
  → onValueChange fires with "J"
  → fieldValue = "J"
  → Compose recomposes the OutlinedTextField (it reads fieldValue)
  → the Text("Hello, $greetingName") does NOT recompose (it doesn't read fieldValue)
```

```
The user taps "Change"
  → onClick fires
  → greetingName = fieldValue (say, "John")
  → Compose recomposes Text("Hello, $greetingName")
  → the OutlinedTextField does NOT need to recompose (it doesn't read
greetingName)
```

Contrast with `ChangeName` (imperative): in the Views version there was exactly one line that updated the UI (`greetingText.text = "Hello, ${nameField.text}"`), written once inside the click listener, and the programmer had to consciously decide *when* to run it. In the Compose version there are **two separate declarations** — `Text(text = "Hello, $greetingName")` and `OutlinedTextField(value = fieldValue, ...)` — and neither one is an instruction to "update now." Each is a standing description of what that piece of UI should show, given the state it reads. Compose decides *when* to re-run each one by watching which state changed — the programmer never writes an update statement at all. This is the core shift shown in section 1: instead of "event → programmer writes code to push the new value into a widget," it's "event → state changes → Compose figures out which descriptions need to run again."

5.4 The central idea

In Compose, the unit of recomposition is not "the whole function" but **each composition scope that actually reads a given piece of state**. Each `Text`, each child composable, recomposes independently based on what state it reads — not because it happens to live in the same function as something that changed.

Activity: before running the app, try to answer:

1. If you add a `Text("Length: ${fieldValue.length}")` right below the `TextField`, how often does it recompose?
2. If you replace `greetingName` with an `Int` that counts how many times "Change" was tapped, and show it in another `Text`, does that new `Text` recompose while you're typing in the field?

Then implement it and check your answers against the real behavior.

6. Golden rules of recomposition

6.1 Never rely on side effects

```
// ❌ Bad: reading SharedPreferences inside the composable
@Composable
fun SharedPrefsToggleBad() {
    val value = sharedPrefs.getBoolean("key", false) // expensive, and may run
many times
    ...
}

// ✅ Good: the value arrives as a parameter, the read happens in the ViewModel
@Composable
fun SharedPrefsToggle(
```

```

    text: String,
    value: Boolean,
    onValueChanged: (Boolean) -> Unit
) {
    Row {
        Text(text)
        Checkbox(checked = value, onCheckedChange = onValueChanged)
    }
}

```

Relate this back to `ChangeName`: the composable doesn't read from or write to anything external (a file, the network, a database); all state lives in `remember` and is updated through callbacks (`onValueChanged`, `onClick`). That's exactly the correct pattern.

6.2 Recomposition is optimistic and can be canceled

- Compose starts recomposing expecting to finish before parameters change again.
- If a parameter changes while recomposition is in progress, Compose **cancels and restarts** with the new value.
- If there were a side effect in the middle, it could end up "applied" even though the composition was discarded → inconsistent state.
- **Conclusion:** idempotence and freedom from side effects are what make canceling/restarting safe.
- **Important distinction:** discarding a canceled composition only throws away the *UI tree* that call would have produced — it does **not** undo anything that already happened outside that call, such as a database write, a mutation to a shared/global object, or a real callback with external consequences. A local variable declared and only read inside that same function call is the *one* case where cancellation is harmless, precisely because nothing about it escapes the call.

6.3 Composable functions run often — avoid expensive work

- They may run on every frame of a UI animation.
- Forbidden example: reading from disk or network directly inside the composable.
- Correct pattern: the expensive read happens in a coroutine in the `ViewModel`, and the result arrives as a parameter/state.

6.4 Never write to local variables during composition

Revisit the snippet from section 2.3:

```

// From the article – looks suspicious, but let's check it carefully
@Composable
fun ListWithBug(myList: List<String>) {
    var items = 0
    Row {
        Column {
            for (item in myList) {
                Card {
                    Text("Item: $item")
                    items++ // side effect of recomposition
                }
            }
        }
    }
}

```

```

    }
  }
  Text("Count: $items")
}
}

```

Important clarification before condemning this code: trace through a single call to `ListWithBug`. `items` is declared fresh inside the function body, so every time Compose calls `ListWithBug` again, `items` restarts at 0, the loop runs to completion, and `items` ends up equal to `myList.size` before `Text("Count: $items")` reads it. If that composition is committed, the count shown is correct. If it's canceled instead, Compose simply doesn't emit anything from that discarded pass — it doesn't leave a stale wrong number on screen. **So this exact snippet does not actually produce a visible bug.**

The real problem is a matter of *habit and principle*, not a bug you can point to on screen here:

- The only reason this is safe is that `items` is freshly re-declared on every single call. Nothing here is "remembered" across recompositions or shared between composables.
- The moment that stops being true — the variable gets hoisted out of the function body, promoted to `remember`, or captured by something that outlives one call — the exact same pattern (`items++` inside a loop, read afterwards) turns into a real, visible bug.

Here is a version where the bug actually appears:

```

// ❌ Really broken: the counter survives across recompositions
@Composable
fun ListWithRealBug(myList: List<String>) {
    var items by remember { mutableIntStateOf(0) }
    Column {
        for (item in myList) {
            Text("Item: $item")
            items++ // adds to the PREVIOUS total instead of recomputing it
        }
    }
    Text("Count: $items")
}

// ✅ Good: no mutable counter at all, just derive the value every time
@Composable
fun ListComposable(myList: List<String>) {
    Column {
        for (item in myList) {
            Text("Item: $item")
        }
        Text("Count: ${myList.size}")
    }
}

```

Now `items` is kept alive across recompositions by `remember`. Every time `ListWithRealBug` recomposes — for any reason, not necessarily because `myList` changed — the loop runs again and adds `myList.size` on top of whatever `items` already was. After a few recompositions with a 3-item list, the displayed count is 6, then 9, then 12... growing without bound, never showing the actual size of the list.

This is also the version where the parallelism warning becomes real: if two recompositions of this composable could ever run concurrently, both would read-then-write the same remembered `items`, racing on it. A plain local `var` reinitialized every call (like in `ListWithBug`) has nothing to race on — there's no shared state to race over.

Takeaway: the rule "don't mutate variables inside a composable" is really about not letting any piece of mutable state *persist or escape* across calls without an explicit mechanism (`remember`, hoisted state, a `ViewModel`) that Compose knows how to manage. A local variable that's declared and consumed within a single call is harmless on its own — but writing that habit into your fingers is what leads directly to bugs like `ListWithRealBug` the first time that variable gets promoted to survive across recompositions.

6.5 Execution order isn't guaranteed

```
@Composable
fun ButtonRow() {
    MyFancyNavigation {
        StartScreen()
        MiddleScreen()
        EndScreen()
    }
}
```

- `StartScreen()`, `MiddleScreen()`, and `EndScreen()` may run in any order.
- You can't assume `StartScreen()` runs first and leaves something ready (a global variable, say) for `MiddleScreen()` to use.
- Every composable must be self-contained. (This applies to a possible future where Compose can run composables in parallel — even though it currently doesn't, code should be written as if it could.)

7. Summary

Compose isn't "magic" — it's a compiler and a runtime that decide, based on what state each composable reads, which parts of the UI tree need to run again. If a composable is fast, idempotent, and side-effect free, that process is safe no matter how many times, in what order, or when Compose decides to run it.

Review questions:

1. What does `@Composable` do?
2. What is recomposition, and what triggers it?
3. Why is `remember` necessary in `Counter()`?
4. Why doesn't typing in the `TextField` in `ChangeName` recompose the greeting `Text`?
5. Name one of the three golden rules of a composable.

Exercise: on the `JetPackChangeName` project, add:

- A character counter that updates live while typing (without pressing "Change").
- A "Reset" button that sets both states (`fieldValue` and `greetingName`) back to the initial value "World".

Write, as a comment in the code, which composable(s) recompose on each interaction (keystroke, "Change", "Reset").

8. Kotlin data types, and building a `JSONArray` of people

Before introducing `LazyColumn` (section 9), we need an actual list to display. This section covers just enough Kotlin data types to define one record, and then how to represent a collection of those records as a `JSONArray/JSONObject` — the classic shape data arrives in from a web API, and a natural stand-in for "real" list data in a classroom demo.

8.1 Basic Kotlin data types

Kotlin's built-in types you'll use immediately:

Kotlin type	Holds	Example
<code>Int</code>	whole number	<code>val age: Int = 20</code>
<code>Double</code>	decimal number	<code>val price: Double = 9.99</code>
<code>String</code>	text	<code>val name: String = "Ada"</code>
<code>Boolean</code>	true/false	<code>val isActive: Boolean = true</code>
<code>List<T></code>	ordered collection of T	<code>val names: List<String> = listOf("Ada", "Bob")</code>

Kotlin infers types, so `val age = 20` is the same as `val age: Int = 20` — the type annotation is optional but worth writing explicitly the first few times, for clarity.

8.2 `data class`: modeling one record

A `data class` is Kotlin's dedicated way to define a plain record type — a bundle of named, typed fields with no manual boilerplate. For this list, one record is one person:

```
data class Person(  
    val id: Int,  
    val name: String,  
    val age: Int  
)
```

Declaring `data class` (instead of plain `class`) gives you, for free:

- A constructor: `Person(id = 1, name = "Ada", age = 30)`
- Readable `toString()`, `equals()`, `hashCode()` — useful when debugging or comparing items.
- `copy()` — e.g. `ada.copy(age = 31)` returns a new `Person` with just `age` changed.

This is the type each row of our `LazyColumn` will eventually render. Note the parallel with earlier state examples: `count` was a bare `Int`; here, each list item is a small structured value instead — but it is still just *data* that composables will read and display.

8.3 From a `List<Person>` to a `JSONArray`

Kotlin/Android already ships the classic `org.json` classes (`JSONObject`, `JSONArray`) — no extra library needed for a simple demo. The idea: each `Person` becomes one `JSONObject`; the whole collection becomes one `JSONArray`.

```
import org.json.JSONArray
import org.json.JSONObject

fun personToJson(person: Person): JSONObject {
    val obj = JSONObject()
    obj.put("id", person.id)
    obj.put("name", person.name)
    obj.put("age", person.age)
    return obj
}

fun peopleToJsonArray(people: List<Person>): JSONArray {
    val array = JSONArray()
    for (person in people) {
        array.put(personToJson(person))
    }
    return array
}
```

Sample data to build the array from:

```
val people = listOf(
    Person(id = 1, name = "Ada", age = 36),
    Person(id = 2, name = "Bob", age = 24),
    Person(id = 3, name = "Cleo", age = 41),
    Person(id = 4, name = "Dan", age = 19)
)

val peopleJson: JSONArray = peopleToJsonArray(people)
println(peopleJson.toString(2)) // pretty-printed, indent = 2
```

Resulting JSON (conceptually — this is what `toString(2)` prints):

```
[
  { "id": 1, "name": "Ada", "age": 36 },
  { "id": 2, "name": "Bob", "age": 24 },
  { "id": 3, "name": "Cleo", "age": 41 },
```

```
{ "id": 4, "name": "Dan", "age": 19 }
]
```

8.4 Going back: `JSONArray` to `List<Person>`

Worth showing the reverse direction too, since this is exactly what happens when real data arrives from a network call:

```
fun jsonToPerson(obj: JSONObject): Person {
    return Person(
        id = obj.getInt("id"),
        name = obj.getString("name"),
        age = obj.getInt("age")
    )
}

fun jsonArrayToPeople(array: JSONArray): List<Person> {
    val result = mutableListOf<Person>()
    for (i in 0 until array.length()) {
        result.add(jsonToPerson(array.getJSONObject(i)))
    }
    return result
}
```

Key points

1. **data class = a typed shape for one row.** Before this, "state" in our examples was a single primitive (`Int`, `String`). A `LazyColumn` needs a *list of structured records* — `data class` is the minimal, idiomatic way to define that shape in Kotlin.
2. **`JSONArray`/`JSONObject` are just a serialization format.** They are not a special Kotlin/Compose concept — they're the generic, stringly-typed way to represent "a list of records" that any web API would return. Converting `List<Person>` → `JSONArray` and back is the same round-trip a real app does when it talks to a server.
3. **This is where the list for section 9 comes from.** We now have `people: List<Person>` (or, equivalently, a `JSONArray` we can parse into one) — this is exactly the list the upcoming `LazyColumn` will render, one row per `Person`.

Try it: run `JetPackListPeople` and check Logcat (`adb logcat -s JetPackListPeople:D`). You should see `peopleJson.toString(2)` printed as pretty-printed JSON, followed by the reconstructed list from `jsonArrayToPeople(peopleJson)`, and confirmation that the round trip reconstructs the original `List<Person>`.

9. Introducing `Lazy` / `LazyColumn`

Motivating problem

Suppose we want to show a list of, say, 5,000 people (the `List<Person>` from section 8, just bigger). With a plain `Column`, **every single row composable is created and measured immediately**, even the ones far off-screen. That's wasteful in memory and CPU, and for a long enough list it will visibly lag or crash.

```
// Don't do this for long lists:
Column(modifier = Modifier.verticalScroll(rememberScrollState())) {
    people.forEach { person -> PersonRow(person) }
}

@Composable
fun PersonRow(person: Person) {
    Text(text = "${person.name} (${person.age})")
}
```

The fix: `LazyColumn`

```
LazyColumn {
    items(people) { person ->
        PersonRow(person)
    }
}
```

`LazyColumn` (and its cousin `LazyRow`) only composes and lays out the items that are **actually visible on screen, plus a small buffer just outside the viewport**. As the user scrolls, items that scroll off are disposed and new items that scroll into view are composed on demand.

The key concept for this section: "smart" recomposition inside `LazyColumn`

This is the deeper idea beyond just "it's more efficient because it doesn't render off-screen items." The important point is what happens on *recomposition*, not just initial composition:

- Each `item { }` (or each row generated by `items(list) { }`) is its **own composable scope**. If the state backing *one* row changes, Compose recomposes **only that row's composable**, not the whole list, and not even the neighboring rows.
- This mirrors exactly what we saw in sections 4 and 5 (state and recomposition with the counter and `ChangeName`) — Compose recomposition is always scoped to "which composable actually reads the state that changed." `LazyColumn` doesn't introduce a new rule; it's the *same* rule, now applied across potentially thousands of rows.
- Practical consequence: if each row has its own `remember { mutableStateOf(...) }` (e.g., a "favorite" checkbox per person, or a per-row counter), tapping row #3742's checkbox recomposes *only* row #3742. Compose does not walk back through the other 4,999 rows.
- Contrast this with what would happen if the whole list were driven by one big `mutableStateOf(List<Person>)` at the top and the list itself, rather than individual rows, needed to change: then any per-item mutation typically means replacing the whole list (since `List` is immutable), and now the *comparison* of old vs. new list matters. This is where `key = { person.id }` in `items(...)` becomes important: it tells Compose "this row corresponds to the same conceptual

Person across recompositions, even if its position in the list changed" — enabling Compose to skip recomposing/reordering rows that didn't actually change, and to preserve each row's own internal **remembered** state correctly when the list is reordered/filtered. This is exactly why **Person** has an **id: Int** field from section 8 — it's the stable identity **key** relies on. (This connects directly to appendix A7 below, which covers **key** in more depth.)

```
LazyColumn {
    items(items = people, key = { person -> person.id }) { person ->
        PersonRow(person)
    }
}
```

Hands-on exercise

1. In **JetPackListPeople**, replace the hardcoded 4-person list with a larger generated **List<Person>** (e.g. `(1..2000).map { Person(id = it, name = "Person $it", age = 18 + it % 50) }`).
2. Add a **Log.d** (or a visible per-row counter with its own **remember**) inside the row composable.
3. Scroll down and back up — check the logs to confirm rows get disposed/recomposed as they leave/enter the viewport, unlike a plain **Column** where everything exists at once.
4. Give each row a per-row **Button** that increments a per-row **remembered** counter. Click one row's button, then scroll away and back — the counter should be preserved (since **key** is stable). Then try removing **key** and see what goes wrong when the list is reordered.

Wrap-up

Composition builds the UI tree. Recomposition keeps only the parts that read *changed* state in sync — automatically, and always scoped to the smallest composable that actually reads that state.

LazyColumn doesn't change this rule; it just applies it at scale, while *also* skipping composition entirely for rows that aren't even on screen yet.

Additional material

Two pieces of vocabulary used throughout sections 4 and 5 — "Compose can **observe** state changes" and "**by** is Kotlin's property **delegation**" — are not Compose-specific jargon. They name two well-known, general-purpose software design patterns that Compose happens to build on.

A1. The Observer pattern

The general pattern: an object (the **subject**) maintains a list of dependents (**observers**) and automatically notifies them whenever its own state changes, without the subject needing to know what those observers are going to do with that information. Classic examples: a spreadsheet cell notifying every formula that references it when its value changes; a GUI button's "click" event notifying every listener registered on it; RxJava/Kotlin **Flow** emitting values to collectors.

How this maps to Compose: `mutableStateOf(0)` returns a **MutableState<Int>** object. That object is the *subject*. It doesn't hold a literal list of registered listener objects the way a textbook Observer implementation might — instead, Compose's runtime uses a mechanism called a **snapshot system**:

- While a composable function is running, Compose keeps track of every piece of observable state it *reads* (every `.value` access, or every `count` access when using `by`). This builds an implicit "this composable depends on this state" relationship — this is the "subscribing" step, done automatically instead of by an explicit `addObserver()` call.
- When that state's value is later *written*, Compose's snapshot system notifies its recomposition scheduler that every composable which previously read that state is now invalid ("dirty") and needs to run again. This is the "notify observers" step.

Analogy: how does a spreadsheet know to recalculate `=A1+A2` when you change `A1`? Nobody manually tells every formula in the sheet to recheck itself — the spreadsheet engine tracks which cells each formula reads, and automatically re-evaluates the formula when one of those cells changes. That is the Observer pattern, and it is functionally the same idea as Compose recomposing `Text(text = "$count")` when `count` changes.

Where the analogy should not be pushed too far: in the classic Observer pattern (as in the GoF book, or Java's old `Observable/Observer` classes), the subject exposes explicit `subscribe/addObserver` and `notifyObservers` methods that application code calls directly. Compose's snapshot system does the equivalent bookkeeping *implicitly*, by instrumenting reads and writes of `.value` during composition — there's no explicit subscribe call anywhere in `Counter()` or `ChangeName()`. It's the same pattern at the conceptual level (subject/observer relationship, automatic notification on change) but a different, more automatic implementation strategy than the pattern's textbook form.

A2. The Delegation pattern and Kotlin's `by`

The general pattern: instead of a class implementing a piece of behavior itself, it holds a reference to another object and forwards ("delegates") the call to that object. The class *appears* to have the behavior from the outside, but doesn't implement it — it hands the work off. Classic example: a `Car` class that delegates `startEngine()` to an `Engine` object it holds a reference to, rather than implementing engine-starting logic inline.

How this maps to Kotlin specifically — property delegation: Kotlin has a language feature, formalized with the `by` keyword, specifically for *delegating a property's getter and setter* to another object, instead of delegating a whole method. Any object used with `by` for a property must provide `getValue()` and (for a `var`) `setValue()` operator functions matching a specific convention. Here is the actual line from `Counter()`, with nothing renamed or added:

```
var count by remember { mutableIntStateOf(0) }
```

Important — there is only one name in this line: `count`. `remember { mutableIntStateOf(0) }` evaluates to a `MutableIntState` object, but that object is never given a name of its own here — it's an anonymous expression on the right of `by`, the same way `2 + 2` in `val x = 2 + 2` never gets its own name either. Kotlin takes that unnamed object and stores it in a hidden backing field that the compiler generates for you (you never see or write its name), and `count` becomes a property whose `get/set` are delegated to it.

- `remember { mutableIntStateOf(0) }` produces "the delegate object" — a `MutableIntState`.
- `MutableIntState` provides `getValue/setValue` operator extension functions (defined by the Compose runtime library) that satisfy Kotlin's delegation convention.

- `var count by <the delegate object>` makes `count` a property whose reads and writes are **delegated** to it: writing `count` actually calls the delegate object's `getValue()`, and writing `count = x` actually calls its `setValue(x)`.

`by` doesn't change *what* happens (it's still ultimately calling the delegate's getter/setter) — it only changes the *syntax*, making a delegated property read and write like an ordinary local variable, with no separate name for the underlying object ever needed. This is purely a readability/ergonomics feature; the underlying Observer-pattern behavior from A1 is exactly the same whether or not `by` is used.

by is optional. You can write the exact same behavior without it, by keeping the `remember { ... }` result in an explicitly named variable instead of delegating to it:

```
// With property delegation (idiomatic) – only one name exists: count
var count by remember { mutableIntStateOf(0) }
...
Text(text = "$count", fontSize = 72.sp)
Button(onClick = { count += 1 }) { ... }

// Without it – same behavior, no delegation, two names now exist: countState and
// its .value
val countState = remember { mutableIntStateOf(0) }
...
Text(text = "${countState.value}", fontSize = 72.sp)
Button(onClick = { countState.value += 1 }) { ... }
```

These two versions behave identically — same recomposition, same triggering event. `countState` in the second version is just an ordinary variable name the programmer chose, exactly like naming any other `val`. There's nothing Compose-specific about that name; it could be `holder`, `x`, or `myCounterBox`.

Takeaway: `remember { mutableStateOf(...) }` is what gives Compose an object it can *observe* (Observer pattern — A1); `by` is purely a Kotlin syntax feature that lets that observed object's value be read and written using ordinary property syntax, without ever needing to give the object its own name (Delegation pattern — A2). The two patterns solve two different problems and happen to be combined in almost every line of Compose state code, which is why they show up together so often that it's easy to mistake them for one single Compose-specific idea.

A3. The same three apps in vanilla JavaScript

Vanilla JavaScript — plain HTML + JS, with no framework like React, Vue, or Svelte — is worth adding as a **third data point**, because the browser's DOM API is imperative in exactly the same sense as Android Views: elements are live, stateful objects, you fetch a reference to one and mutate its properties by hand, and nothing re-runs automatically when your data changes.

Display0 — vanilla JS:

```
<!DOCTYPE html>
<html>
<body>
```

```

<div style="display:flex; justify-content:center; align-items:center;
height:100vh;">
  <span id="numberText" style="font-size:72px;">0</span>
</div>
</body>
</html>

```

There is no JavaScript file at all needed for this one — same as `Display0's MainActivity.kt`, which only calls `setContentView` and never touches `numberText` again. The "0" is static HTML content, exactly the way it was a static `android:text="0"` attribute in XML.

IncreasingCounter — vanilla JS:

```

<!DOCTYPE html>
<html>
<body>
  <div style="display:flex; flex-direction:column; align-items:center; justify-
content:center; height:100vh;">
    <span id="counterText" style="font-size:72px;">0</span>
    <button id="addButton" style="margin-top:16px;">Add+1</button>
  </div>

  <script>
    let count = 0;

    const counterText = document.getElementById("counterText");
    const addButton = document.getElementById("addButton");

    addButton.addEventListener("click", () => {
      count += 1;
      counterText.textContent = count;
    });
  </script>
</body>
</html>

```

ChangeName — vanilla JS:

```

<!DOCTYPE html>
<html>
<body>
  <div style="display:flex; flex-direction:column; align-items:center; justify-
content:center; height:100vh;">
    <span id="greetingText" style="font-size:24px;">Hello, World</span>
    <input id="nameField" type="text" value="World" style="margin-top:16px;">
    <button id="changeButton" style="margin-top:16px;">Change</button>
  </div>

  <script>

```

```

    const initialName = "World";

    const greetingText = document.getElementById("greetingText");
    const nameField = document.getElementById("nameField");
    const changeButton = document.getElementById("changeButton");

    greetingText.textContent = "Hello, " + initialName;
    nameField.value = initialName;

    changeButton.addEventListener("click", () => {
        greetingText.textContent = "Hello, " + nameField.value;
    });
</script>
</body>
</html>

```

Line-by-line correspondence table:

Concept	Android Views (Kotlin)	Vanilla JavaScript
Get a reference to a live widget/element	<code>findViewById<TextView>(R.id.counterText)</code>	<code>document.getElementById("counterText")</code>
A widget's own internal state	<code>counterText.text</code>	<code>counterText.textContent</code>
An editable field's current value	<code>nameField.text</code>	<code>nameField.value</code>
Register a callback for a tap/click	<code>addButton.setOnClickListener { ... }</code>	<code>addButton.addEventListener("click", () => { ... })</code>
App-level state, invisible to any widget	<code>private var count = 0</code> (an <code>Activity</code> field)	<code>let count = 0</code> (a variable in the enclosing script scope)
Pushing a changed value into the UI	<code>counterText.text = "\$count"</code> (written by hand)	<code>counterText.textContent = count</code> (written by hand)

Every single row of that table is something the *programmer* does by writing a line of code that says "go update this specific element now." Nothing in vanilla JS or in Android Views watches `count` and decides on its own that `counterText` needs to change. This is also historically accurate: this is exactly the pain point that motivated the creation of frameworks like React (open-sourced in 2013) in the frontend world, which introduced the same declarative, "UI as a function of state," automatic-re-render idea that Jetpack Compose (stable release in 2021, citing React and similar frameworks as an influence) later brought to Android. **Vanilla JS today is where Android Views has always been; React (and other modern JS frameworks) is where Compose is.**

If you've used React, Vue, or Svelte before: think about how that framework decides when to re-render a piece of UI. Every mainstream answer maps onto the same two ideas covered in this document — some notion of observable/reactive state, and some mechanism for automatically re-running a UI description when that state changes. The vocabulary differs (React calls it "re-rendering a component," Vue calls it "reactivity," Compose calls it "recomposition") but the underlying idea — and the imperative pain it's solving — is the same.

A4. The same three apps in React

React is the frontend framework that popularized the declarative, "describe the UI as a function of state, and let the framework re-render when that state changes" model, and it's the framework most often named as an influence on Compose's design.

JetPackDisplay0 → React DisplayZero:

```
function DisplayZero() {
  return (
    <div style={{ display: "flex", justifyContent: "center", alignItems: "center",
height: "100vh" }}>
      <span style={{ fontSize: "72px" }}>0</span>
    </div>
  );
}
```

Same as the Compose version: no state, no event handlers, nothing that could ever cause a re-render.

JetPackIncreasingCounter → React Counter:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div style={{ display: "flex", flexDirection: "column", alignItems: "center",
justifyContent: "center", height: "100vh" }}>
      <span style={{ fontSize: "72px" }}>{count}</span>
      <button onClick={() => setCount(count + 1)}>Add+1</button>
    </div>
  );
}
```

JetPackChangeName → React ChangeName:

```
import { useState } from "react";

function ChangeName() {
  const initialName = "World";
```

```

const [greetingName, setGreetingName] = useState(initialName);
const [fieldValue, setFieldValue] = useState(initialName);

return (
  <div style={{ display: "flex", flexDirection: "column", alignItems: "center",
justifyContent: "center", height: "100vh", padding: "16px" }}>
    <span style={{ fontSize: "24px" }}>Hello, {greetingName}</span>
    <input
      value={fieldValue}
      onChange={(e) => setFieldValue(e.target.value)}
      style={{ marginTop: "16px" }}
    />
    <button onClick={() => setGreetingName(fieldValue)} style={{ marginTop:
"16px" }}>
      Change
    </button>
  </div>
);
}

```

Line-by-line correspondence table — Compose vs. React:

Concept	Jetpack Compose (Kotlin)	React (JSX)
Mark a function as UI-describing	<code>@Composable</code> annotation	No annotation needed — any function returning JSX is a "component" by convention (capitalized name)
Create an observable piece of state	<code>remember { mutableStateOf(0) }</code>	<code>useState(0)</code>
Read the current value	<code>count</code> (via <code>by</code> delegation) or <code>count.value</code>	<code>count</code> (the first element of the pair <code>useState</code> returns)
Update the value and trigger a re-run	<code>count = it / count += 1</code>	<code>setCount(newValue)</code> — you never assign to <code>count</code> directly
What gets re-run when state changes	The composable function (or the smallest scope inside it that read the state) — recomposition	The component function — re-render
Declare a piece of UI	Calling <code>Text(...)</code> , <code>Button(...)</code> , etc. inside the function body	Returning JSX (<code></code> , <code><button></code> , etc.) from the function
Wire up a click	<code>Button(onClick = { ... })</code>	<code><button onClick={() => { ... }}></code>
Two-way-bound text field	<code>OutlinedTextField(value = fieldValue, onChange = { fieldValue = it })</code>	<code><input value={fieldValue} onChange={ (e) => setFieldValue(e.target.value)} /></code>

The one structural difference worth calling out explicitly: in Compose, `count += 1` (via `by`) looks like a plain variable assignment, but — as covered in A2 — it's secretly calling `MutableState.setValue()` through Kotlin's property delegation. In React, `useState` doesn't hide this at all: it returns a `[value, setValue]` pair, and `setCount(...)` is an explicit, visible function call — there is no way to "accidentally" look like you're mutating `count` directly, because plain JavaScript has no equivalent to Kotlin's `by`. **Functionally the two are doing the exact same thing (call a setter that schedules a re-run); Compose's `by` just makes that setter call invisible at the call site, while React's `setCount` keeps it visible.** The setter-call-that-triggers-a-rerun is the Observer-pattern behavior (present in both); `by` hiding it behind ordinary assignment syntax is the Delegation-pattern layer on top (Compose-specific, absent in React).

The field/greeting split in `ChangeName` demonstrates the same "skip as much as possible" idea in React too: typing in the `<input>` only calls `setFieldValue`, which only causes whatever part of the component tree reads `fieldValue` to re-render — in a component this small React re-renders the whole `ChangeName` function either way, but in a larger component tree, React has its own optimizations (memoization via `React.memo`, `useMemo`, `useCallback`) to avoid re-rendering subtrees that don't depend on the changed state, conceptually parallel to Compose's per-scope skipping described in section 5.4. The mechanism differs in detail, but the motivating problem — "don't redo work for UI that doesn't depend on what changed" — is identical.

Try it yourself: if you have access to React DevTools, turn on "highlight updates when components render" (or just add a `console.log` at the top of `Counter`) while clicking "Add+1", and compare what you see to the recomposition trace in section 4.4. Both should show the same pattern: the piece of UI reading the changed state re-runs, the button around it does not need to.

A5. The same three apps in SwiftUI

SwiftUI is Apple's own declarative UI framework for iOS/macOS/watchOS/tvOS — released in 2019, and one of the frameworks most frequently mentioned (alongside React) as a peer/influence in discussions of Compose's design. It's a particularly close comparison to Compose because both target native mobile UI (unlike React, which targets the browser) and both use a similar "properties that hold state, and a declarative body that's automatically re-evaluated when that state changes" structure.

JetPackDisplay0 → SwiftUI DisplayZero:

```
struct DisplayZero: View {
    var body: some View {
        Text("0")
            .font(.system(size: 72))
    }
}
```

JetPackIncreasingCounter → SwiftUI CounterView:

```
struct CounterView: View {
    @State private var count = 0

    var body: some View {
        VStack {
```

```
        Text("\($count)")
            .font(.system(size: 72))
        Button("Add+1") {
            count += 1
        }
    }
}
```

JetPackChangeName → SwiftUI ChangeNameView:

```
struct ChangeNameView: View {
    @State private var greetingName = "World"
    @State private var fieldValue = "World"

    var body: some View {
        VStack {
            Text("Hello, \($greetingName)")
                .font(.system(size: 24))
            TextField("Name", text: $fieldValue)
                .padding(.top, 16)
            Button("Change") {
                greetingName = fieldValue
            }
                .padding(.top, 16)
        }
        .padding()
    }
}
```

Line-by-line correspondence table — Compose vs. SwiftUI:

Concept	Jetpack Compose (Kotlin)	SwiftUI (Swift)
Mark something as UI-describing	@Composable annotation on a function	Conforming a struct to the View protocol
Create an observable piece of state	remember { mutableStateOf(0) }	@State private var count = 0
Read the current value	count (via by delegation)	count (via the @State property wrapper — a different language mechanism, but the same effect: reading count secretly reaches into framework-managed storage)
Update the value and trigger a re-run	count = it / count += 1	count += 1 (also looks like plain assignment)

Concept	Jetpack Compose (Kotlin)	SwiftUI (Swift)
What gets re-run when state changes	The composable function (or the smallest scope inside it that read the state) — recomposition	The body property is recomputed — SwiftUI's own internal diffing decides what actually needs to redraw
Declare a piece of UI	Calling <code>Text(...)</code> , <code>Button(...)</code> , etc. inside the function body	Returning <code>Text(...)</code> , <code>Button(...)</code> , etc. from the body property, using Swift's <code>@ViewBuilder</code> (implicit here, same mechanism as Compose's trailing-lambda DSL)
Wire up a tap	<code>Button(onClick = { ... }) { Text("Add+1") }</code>	<code>Button("Add+1") { ... }</code>
Two-way-bound text field	<code>OutlinedTextField(value = fieldValue, onValueChange = { fieldValue = it })</code>	<code>TextField("Name", text: \$fieldValue)</code> — the <code>\$</code> prefix creates a Binding automatically

The most interesting comparison point — **by** (Compose) vs. **@State** (SwiftUI) vs. **useState** (React):

SwiftUI's `@State` and Kotlin's `by remember { mutableStateOf(...) }` solve the *exact* same ergonomics problem the *exact* same way:

- Kotlin's **by** is *property delegation* (a general-purpose language feature, covered in A2) applied to a `MutableState` object that Compose happens to provide.
- Swift's `@State` is a *property wrapper* (a similarly general-purpose Swift language feature — the mechanism is literally named that in the Swift language reference) applied to a value that SwiftUI happens to manage.
- Both let you write `count += 1` and have it look like a plain variable mutation, while secretly calling into a framework-managed piece of observable state underneath — no visible setter call at all.
- React's `useState`, by contrast, refuses to hide this: `setCount(...)` is always an explicit function call.

So on the specific question of "does the state-update call look like a plain assignment, or an explicit function call?", **Compose and SwiftUI agree with each other and disagree with React:**

	Compose (by)	SwiftUI (@State)	React (useState)
State-update syntax	<code>count += 1</code> (looks like assignment)	<code>count += 1</code> (looks like assignment)	<code>setCount(count + 1)</code> (explicit call)
Language mechanism used	Property delegation (Kotlin)	Property wrappers (Swift)	Plain function/array destructuring — JavaScript does have accessor properties (<code>get/set</code> on objects), but nothing with by 's or <code>@propertyWrapper</code> 's "attach this to a variable binding automatically" convention, and <code>useState</code> doesn't use accessor properties at all

	Compose (by)	SwiftUI (@State)	React (useState)
What's hidden underneath	A call into <code>MutableState.setValue()</code>	A call into SwiftUI's internal state storage	Nothing — it's already explicit

Takeaway: whether a state update "looks like" a plain assignment or an explicit function call is a *language feature* decision (does the language have property delegation or property wrappers available to hide it), not a difference in what conceptually happens at runtime. All three frameworks are doing the same Observer-pattern thing underneath (A1): something is being watched, and something gets automatically re-run when it changes. Compose and SwiftUI both chose to spend a language feature to make that watching invisible at the call site; React chose not to (partly because JavaScript, even with modern additions, has no equivalent language feature to spend).

A6. The same three apps in Flutter

Flutter is Google's own cross-platform UI framework (Android, iOS, web, desktop), written in Dart. It's a natural fourth entry in this comparison because, like Compose and SwiftUI, it targets native app UI directly rather than the browser — but on the specific question this section keeps returning to (does the state-update call look like a plain assignment or an explicit function call?), Flutter's `setState` sides with React, not with Compose/SwiftUI.

JetPackDisplay0 → Flutter DisplayZero:

```
class DisplayZero extends StatelessWidget {
  const DisplayZero({super.key});

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text('0', style: TextStyle(fontSize: 72)),
    );
  }
}
```

JetPackIncreasingCounter → Flutter CounterWidget:

```
class CounterWidget extends StatefulWidget {
  const CounterWidget({super.key});

  @override
  State<CounterWidget> createState() => _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
  int count = 0;
```

```

@override
Widget build(BuildContext context) {
  return Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      Text('$count', style: const TextStyle(fontSize: 72)),
      ElevatedButton(
        onPressed: () {
          setState(() {
            count += 1;
          });
        },
        child: const Text('Add+1'),
      ),
    ],
  );
}

```

JetPackChangeName → **Flutter ChangeNameWidget:**

```

class ChangeNameWidget extends StatefulWidget {
  const ChangeNameWidget({super.key});

  @override
  State<ChangeNameWidget> createState() => _ChangeNameWidgetState();
}

class _ChangeNameWidgetState extends State<ChangeNameWidget> {
  static const initialName = 'World';
  String greetingName = initialName;
  String fieldValue = initialName;

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text('Hello, $greetingName', style: const TextStyle(fontSize: 24)),
        TextField(
          controller: TextEditingController(text: fieldValue),
          onChanged: (value) {
            setState(() {
              fieldValue = value;
            });
          },
        ),
        ElevatedButton(
          onPressed: () {
            setState(() {
              greetingName = fieldValue;
            });
          },
        ),
      ],
    );
  }
}

```

```

    });
  },
  child: const Text('Change'),
),
],
);
}
}

```

Line-by-line correspondence table — Compose vs. Flutter:

Concept	Jetpack Compose (Kotlin)	Flutter (Dart)
Mark something as UI-describing	<code>@Composable</code> annotation on a function	Extending <code>StatelessWidget</code> or <code>StatefulWidget</code> and overriding <code>build()</code>
Create a piece of state that survives recomposition	<code>remember { mutableStateOf(0) }</code>	A plain field (<code>int count = 0</code>) on a separate <code>State<T></code> object that Flutter keeps alive across rebuilds
Read the current value	<code>count</code> (via <code>by</code> delegation)	<code>count</code> (a completely ordinary field read — no property wrapper or delegation involved)
Update the value and trigger a re-run	<code>count = it / count += 1</code> (looks like plain assignment; secretly calls a setter)	<code>setState(() { count += 1; })</code> — mutating the field alone does not trigger a rebuild; you must explicitly wrap the mutation in <code>setState</code>
What gets re-run when state changes	The composable function (or the smallest scope inside it that read the state) — recomposition	The widget's <code>build()</code> method — Flutter calls this "rebuilding the widget"
Declare a piece of UI	Calling <code>Text(...)</code> , <code>Button(...)</code> , etc. inside the function body	Returning <code>Text(...)</code> , <code>Column(...)</code> , etc. from <code>build()</code>
Wire up a tap	<code>Button(onClick = { ... })</code>	<code>ElevatedButton(onPressed: () { ... })</code>
Two-way-bound text field	<code>OutlinedTextField(value = fieldValue, onChange = { fieldValue = it })</code>	<code>TextField(controller: ..., onChanged: (value) { ... })</code>

The key structural difference — Flutter does NOT hide the "notify" step: in Compose (`by`) and SwiftUI (`@State`), simply writing `count += 1` is enough — the property wrapper/delegation machinery secretly calls the framework's setter for you, so the mutation *is* the notification. In Flutter, this is **not** true: `count += 1` on its own, outside of `setState`, mutates the field but never tells Flutter to rebuild anything — the screen would silently go stale. `setState(() { ... })` is Flutter's explicit, visible equivalent of "I just changed something, please rebuild" — structurally the same idea as React's `setCount(...)`, just wrapping a block of code instead of taking a new value directly.

Four-way summary table:

	Compose (by)	SwiftUI (@State)	React (useState)	Flutter (setState)
State-update syntax	<code>count += 1</code> (looks like assignment)	<code>count += 1</code> (looks like assignment)	<code>setCount(count + 1)</code> (explicit call)	<code>setState(() { count += 1; })</code> (explicit call wrapping the mutation)
Is the "notify" step hidden?	Yes — hidden behind property delegation	Yes — hidden behind a property wrapper	No — always an explicit call	No — always an explicit call
What happens if you forget the framework- specific part	Impossible to forget — assignment always goes through the delegate	Impossible to forget — assignment always goes through the wrapper	<code>count</code> itself can't be reassigned directly — <code>const [count, setCount] = useState(...)</code> makes <code>count</code> a <code>const</code> , so <code>count = count + 1</code> is a compile error, not a silent bug. The real footgun is different: forgetting to call <code>setCount</code> at all in a handler (nothing happens, screen just doesn't update), or mutating an object/array <i>returned by</i> <code>useState</code> in place instead of replacing it (e.g. <code>list.push(x)</code> instead of <code>setList([...list, x])</code>) — React won't notice the mutation happened	Nothing updates on screen; <code>count</code> changes on the field but <code>build()</code> never re-runs, because mutating a plain field compiles and runs fine on its own

Why this matters: a Compose or SwiftUI developer *cannot* forget to notify the framework, because the language feature makes that impossible by construction; a Flutter developer *can*, because `count += 1` on a plain mutable field compiles and runs fine while silently never triggering a rebuild — a live, real version of exactly the kind of "state changed but nobody told the UI" bug shown throughout sections 1, 4, and 5 for imperative Views code. React's version of "forgetting" is not quite the same shape, though: because `count` is bound with `const`, directly reassigning it is a compile error, not a silent runtime bug — the closest real equivalent in React is forgetting to call `setCount` inside an event handler at all (which just means the handler does nothing visible), or in-place-mutating an object/array held in state instead of replacing it (a more advanced but very real React bug, since React only detects a *new* value was passed to `setX`, not that an old one changed shape).

Note on React Native: React Native is React's component model (`useState`, JSX, functions returning UI) running against a different renderer — native `View/Text/TextInput` components instead of DOM elements — but the state-management story is otherwise identical to A4: `setState`-style updates are just as explicit, and there is no property-wrapper or delegation magic. It isn't covered separately here because it wouldn't add anything to the comparison beyond what Flutter already shows: a framework that targets native mobile UI while still keeping the "notify" step fully explicit.

A7. LazyColumn/LazyRow and the key parameter

This section goes beyond the three example apps in this document, and beyond lists in general — it answers a question that comes up naturally once you start using LazyColumn/LazyRow (Compose's scrollable-list composables) for real data: **if you reorder a list — or reorder it and insert a few new items — does Compose recompose every visible item, or does it reuse what it can?**

The answer depends entirely on whether `items(...)` is given a `key`.

Without a key (the default):

```
LazyColumn {
    items(myList) { item ->
        ItemRow(item)
    }
}
```

Each item's identity inside the lazy list is tied to its **position (index)**, not its content. If the list is reordered, Compose compares old-index-0 vs. new-index-0, old-index-1 vs. new-index-1, and so on — from its point of view, "the item at index 0 changed from A to B." Since the composable's input changed at every reordered position, **every composable at a changed position recomposes**. For a full reversal or shuffle, that's effectively the entire visible list. Inserting an item near the front is even worse: every item after the insertion point shifts by one position, so every one of them looks "changed" and recomposes too — even though none of them conceptually changed at all.

With a stable key:

```
LazyColumn {
    items(myList, key = { item -> item.id }) { item ->
        ItemRow(item)
    }
}
```

Now each item is identified by its `key`, not its position. This turns the diffing into genuine **keyed list reconciliation** — the same concept, and the same reason, behind React's `key` prop on list items (see A4). Given an old list `[A, B, C]` and a new list `[C, D, A, B]` (reordered, with `D` newly inserted):

- **Items whose key is still present (A, B, C):** their composable instances are **reused and moved** to their new position. They are **not recomposed**, and any `remembered` state inside them survives the move intact.
- **The new item (D):** its key didn't exist before, so there's no existing composable to reuse — Compose composes it for the first time at its new position. That's an *initial* composition, not a recomposition of something that already existed.
- Nothing else is touched.

Takeaway: `key` is what lets "recomposition skips as much as possible" (the core idea from section 5.4) hold up even when a list's *order* changes, not just when individual values change. Without a `key`, Compose has no way

to distinguish "this item moved" from "this item's content changed," so it falls back to the conservative, expensive assumption — recompose everything at every position that looks different. This is also why **key** matters beyond raw performance: it's what lets **LazyColumn** correctly preserve scroll position and per-item state (like a checkbox's checked state) when the underlying data reorders.